Singly Linked List (continued) :→

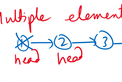
(i) Add first ✓
(ii) Add last ✓
(iii) remove first ✓
(iv) remove last
(v) remove target

{ While adding a node, we should create a new node. *** }
{ While removing / deleting a node, we do not create new nodes, we remove nodes from existing list *** }

* removeFirst ( ) { (i) Empty → (can't remove) message (head == null)

(ii) Simple element:

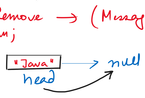
(ii) Multiple elements: →



* Remove the last/tail/end node of the list:

(i) Empty list → Can't Remove → (Message to user) return;

(ii) Single element →




```
if (head.next == null)
{
    head = null;
    size--;
    return;
}
```

(iii) Multiple elements:→

Two pointer approach:→

```
Node secondLast = head;
Node last = head.next;
While (last.next != null) {
    secondLast = last;
    last = last.next;
}
secondLast.next = null;
size--;
```

* Delete / Remove Target Node:→

(i) Empty → just give a message & return

(ii) What if target is the head itself?

delete(1) → target

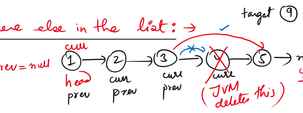
```
if (head.data == target)
{ head = head.next;
  size--;
  return
```

(iii) Target is anywhere else in the list: →

target = 4;

Two pointers:→

```
Node prev = null;
Node curr = head;
While (curr != null && !curr.data.equals(target)) {
    prev = curr;
    curr = curr.next;
}
(check for null) ***
if (curr == null)
sout → target not found
```

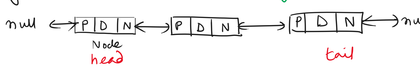


(if found it)

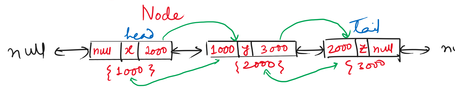
```
prev.next = curr.next;
```

(DRY RUN)

Doubly Linked List :→ (Both forward & Backward traversals)

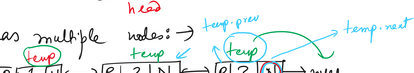


head → tail
tail → head

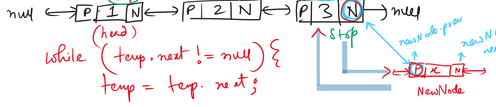


(i) DLL: Empty → (insert at End)

the newNode becomes the head:



(ii) It has multiple nodes: →




```
while (temp.next != null) {
    temp = temp.next;
}
```

1st Connection → temp.next = newNode;
2nd Connection → newNode.prev = temp.next;

null ⟷ 1 ⟷ 2 ⟷ 3 ⟷ null

Linked List's Most Important Interview Questions :→

1. Reverse a linked list ✓
2. Reverse a linked list in groups of k
3. Merge two sorted linked lists
4. Add two numbers using linked lists
5. Find Intersection Point of two linked lists
6. Count number of nodes of a linked list.
7. Floyd's Cycle Detection (Hare & Tortoise Algorithm)
   - (a) Find the middle of a linked list
   - (b) Delete the middle of a linked list
   - (c) Cycle detection in a linked list ✓
8. Even / Odd Filtering of a linked list.